Rapport de la Semaine 1

Architecture Microservices pour la Gestion de Flotte

Équipe Projet :

Zenati Kenza
Mokhtari Celia

Date : 15 mars 2026

Table des matières

1 Introduction **2**

2 Architecture Microservices **3**
2.1 Diagrammes C4 ... 5
2.1.1 Niveau 1 : ... 5
2.1.2 Niveau 2 : ... 6
2.1.3 Niveau 3 : ... 7
2.2 Décisions Architecturales (ADR) ... 8

3 Schémas de Base de Données **9**
3.1 PostgreSQL ... 10
3.1.1 Table `vehicles` ... 10
3.1.2 Exemple de Requête SQL ... 10
3.1.3 Migrations avec Flyway ... 10

4 Contrats d'API **11**
4.1 REST (OpenAPI/Swagger) ... 11
4.1.1 Endpoints Principaux ... 11
4.1.2 Exemple de Contrat OpenAPI ... 11
4.1.3 GraphQL ... 11
4.1.4 Exemple : Schéma `vehicle-service` ... 12

5 Configuration Keycloak **13**
5.1 Étapes de Configuration ... 13
5.1.1 Création du Compte Admin ... 13
5.1.2 Création du Realm `GestionFlotteM1` ... 13
5.1.3 Définition des Rôles (RBAC) ... 13
5.1.4 Création d'un Utilisateur dans `GestionFlotteM1` ... 13
5.2 Test d'Authentification et Validation RBAC ... 14
5.2.1 Requête d'Obtention du Token (cURL) ... 14
5.2.2 Résultat et Décodage du Token ... 14

6 Topics Kafka **17**
6.1 Liste des Topics ... 17
6.2 Exemple de Format de Message (JSON) ... 17
6.2.1 Script pour Créer les Topics ... 17

7 Conclusion **18**
7.1 Bilan de la Semaine 1 ... 18

8 Annexes **19**
8.1 Livrables ... 19

Chapitre 1

Introduction

Ce rapport documente les travaux réalisés durant la **première semaine** du projet de gestion de flotte, couvrant :

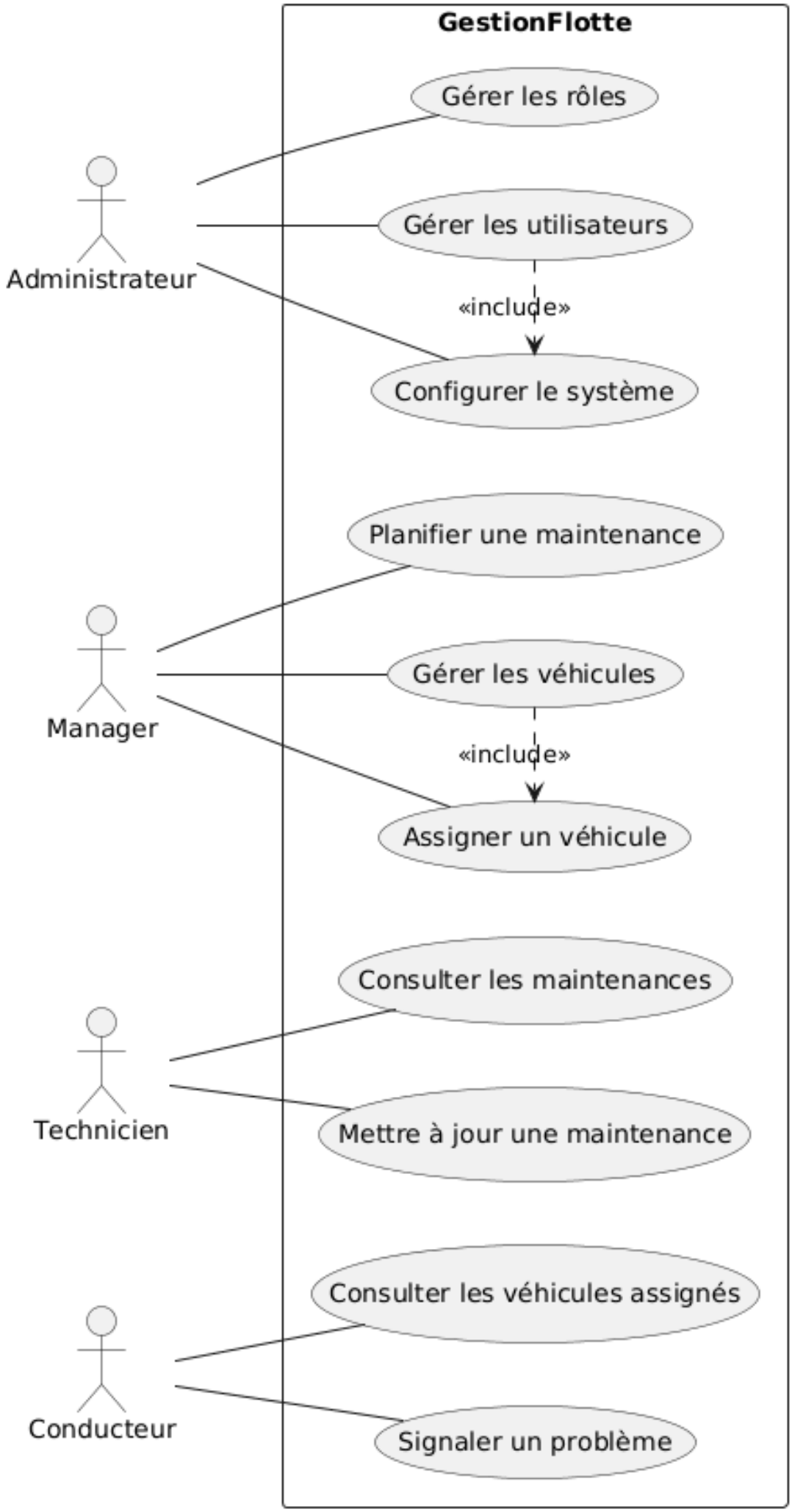
- La finalisation de l'architecture microservices (diagrammes C4, ADR).
- La conception des schémas de base de données (PostgreSQL avec migrations).
- La définition des contrats d'API (OpenAPI/Swagger pour REST, schémas GraphQL).
- La configuration de Keycloak pour l'authentification et les rôles (RBAC).
- La définition des topics Kafka et formats de messages (JSON).

Chapitre 2

Architecture Microservices

Diagramme de Cas d'Utilisation (Use Case)

Ce diagramme montre les interactions entre les acteurs (utilisateurs) et le système. Il est utile pour clarifier les fonctionnalités principales.


GestionFlotte
Gérer les rôles
Gérer les utilisateurs
«include»
Configurer le système
Administrateur
Planifier une maintenance
Gérer les véhicules
«include»
Assigner un véhicule
Manager
Consulter les maintenances
Mettre à jour une maintenance
Technicien
Consulter les véhicules assignés
Signaler un problème
Conducteur

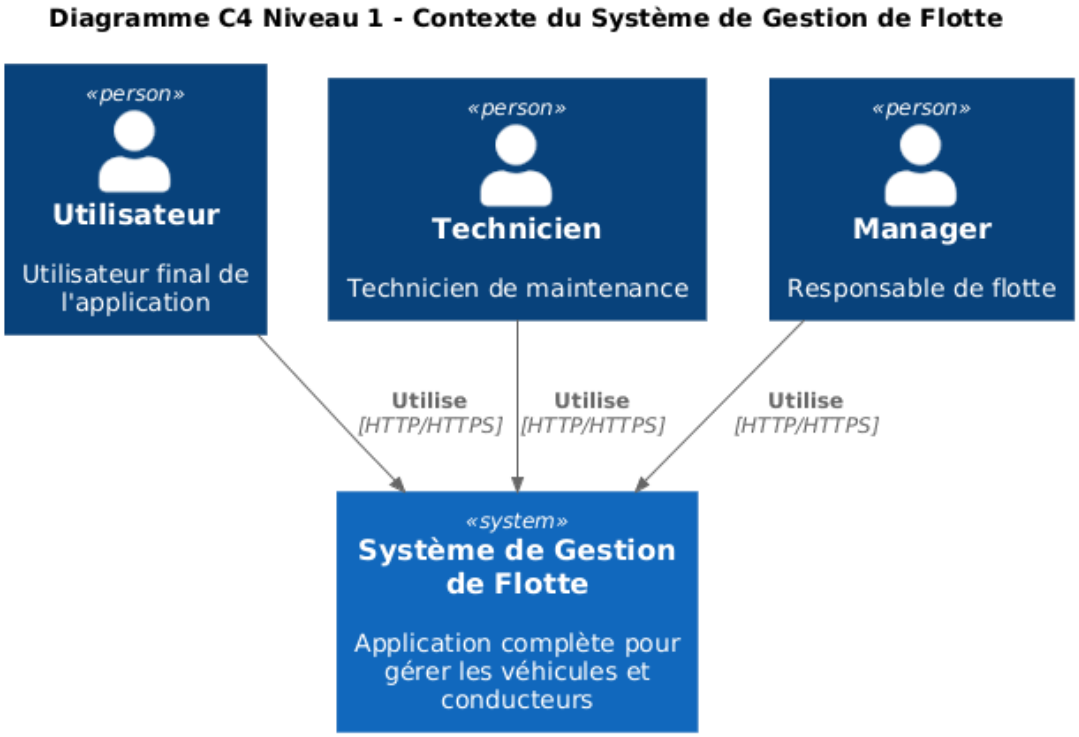
Explications

- **Acteurs :** Administrateur, Manager, Technicien, Conducteur.
- **Cas d'utilisation :**
  - Gestion des utilisateurs/roles (Administrateur)
  - Gestion des véhicules et assignation (Manager)
  - Maintenance (Technicien)
  - Consultation des véhicules (Conducteur)

2.1 Diagrammes C4

2.1.1 Niveau 1 :

Description : *Ce niveau montre le système dans son ensemble et ses interactions avec les utilisateurs et systèmes externes.*



2.1.2 Niveau 2 :

Description : *Ce niveau montre les conteneurs (applications, bases de données, services) qui composent le système.*

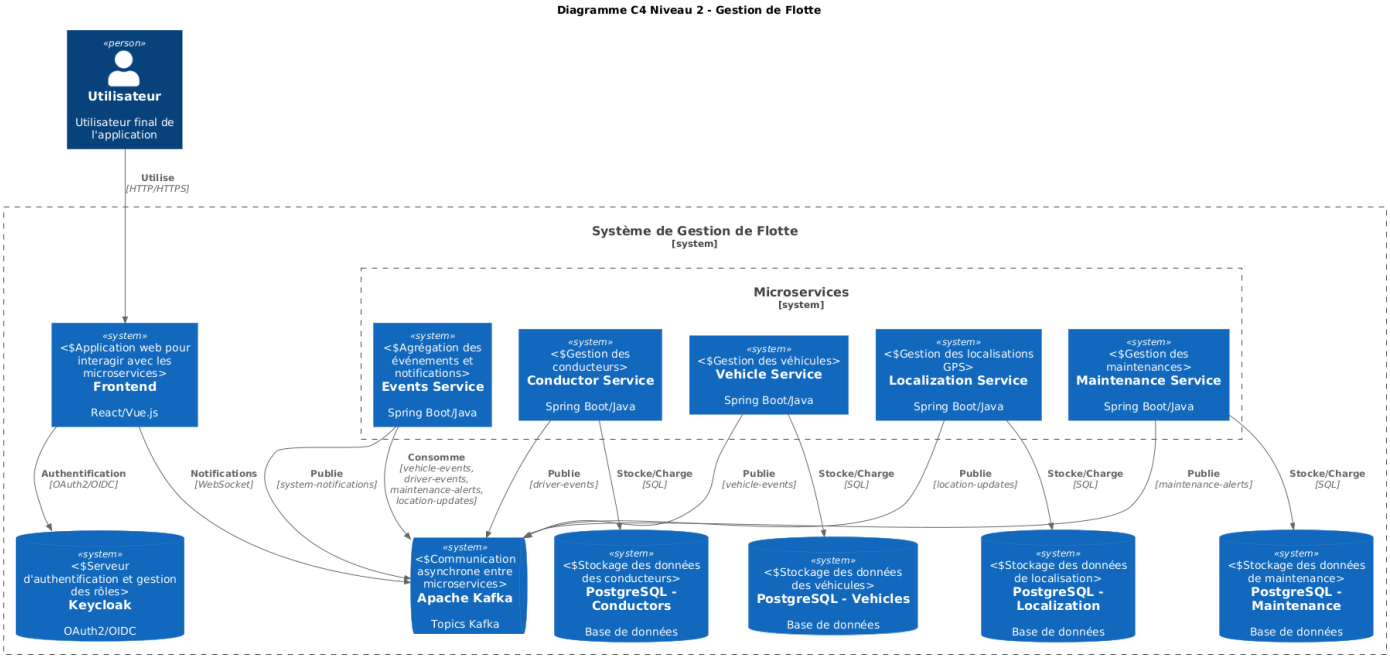
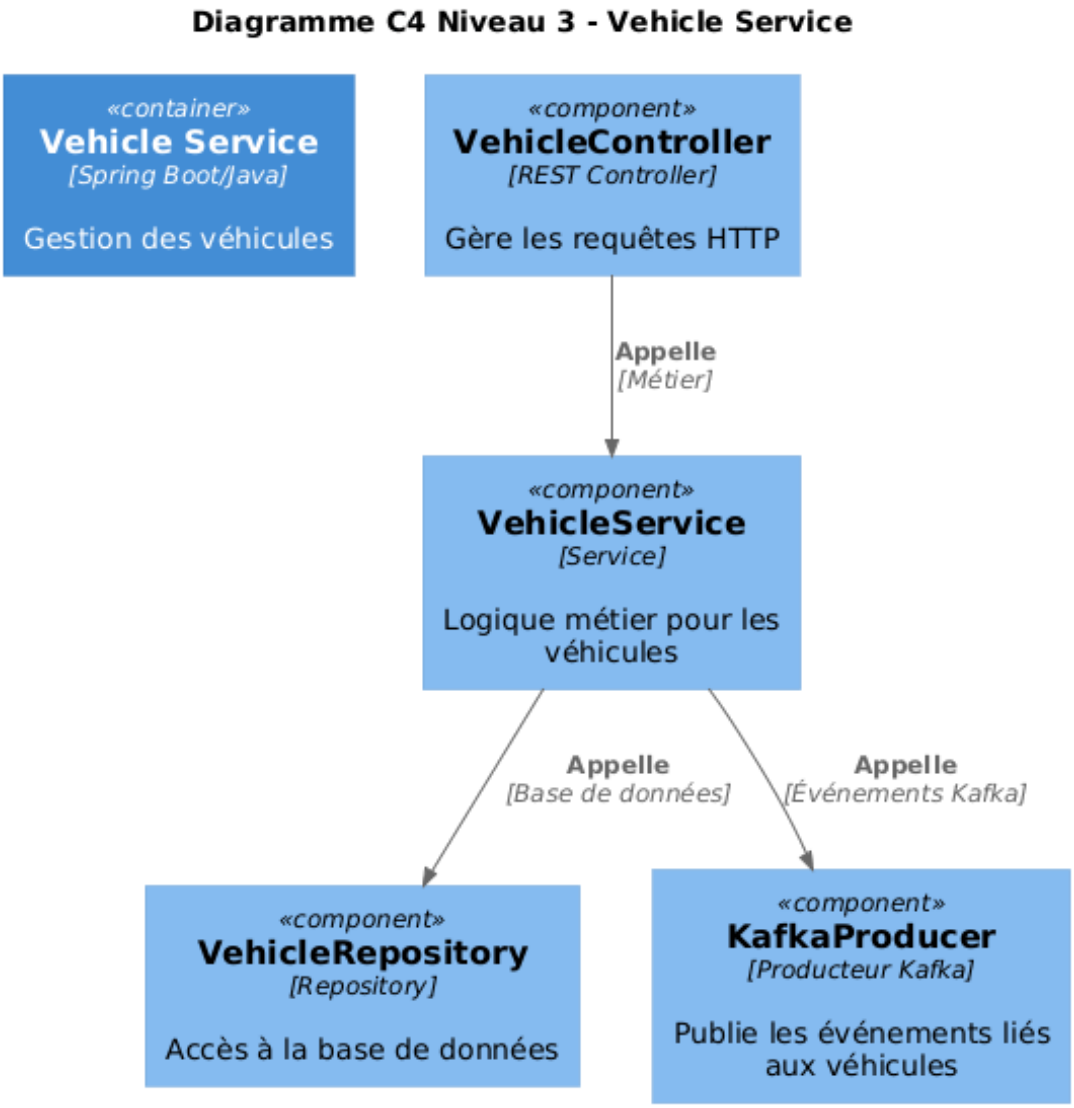
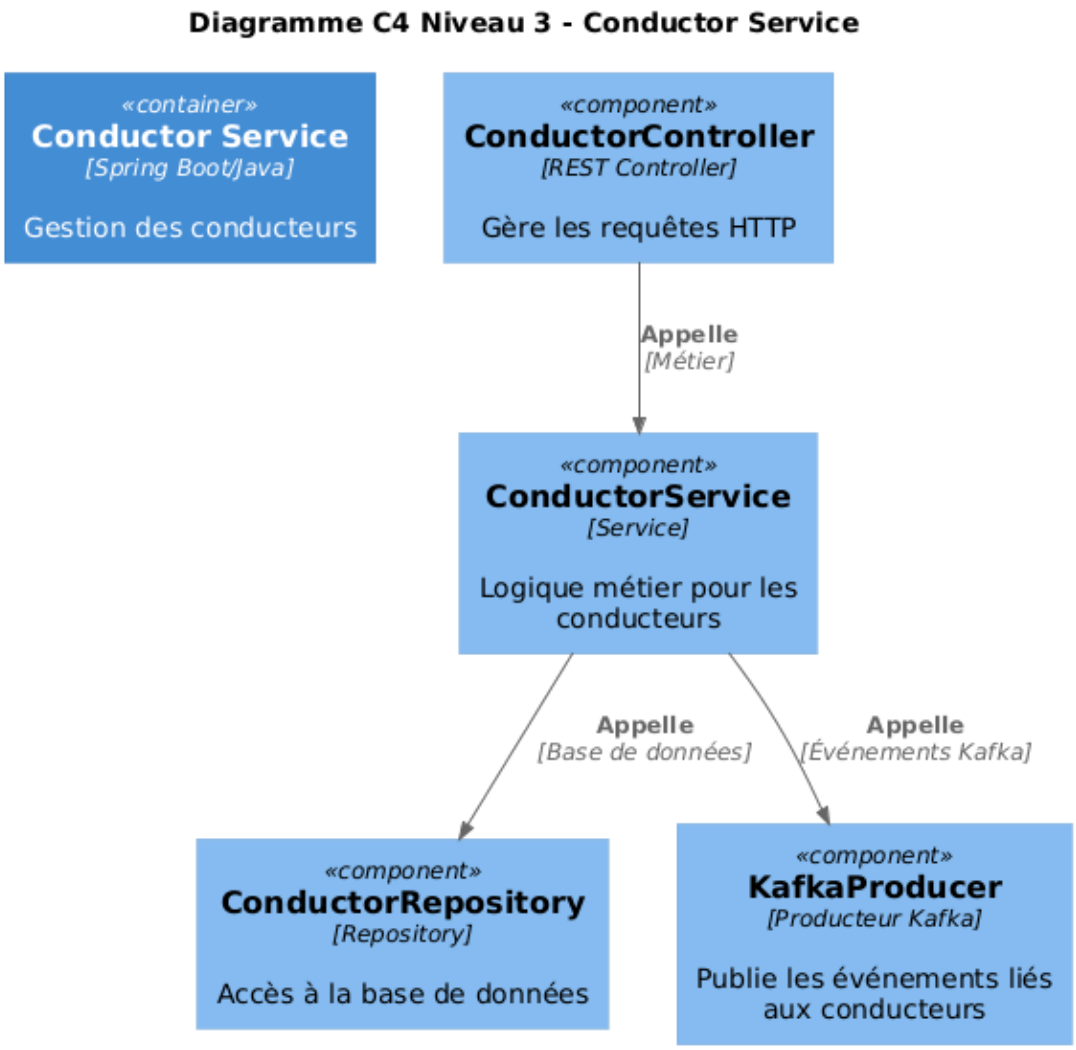


FIGURE 2.1 – Diagramme C4 Niveau 2

2.1.3 Niveau 3 :

Description : *Ce niveau montre les composants internes d'un conteneur. Voici deux exemples pour le microservice vehicle-service et le microservice conductor-service.*





2.2 Décisions Architecturales (ADR)

Les décisions architecturales clés sont résumées dans le tableau ci-dessous. Les fichiers détaillés sont disponibles dans le dossier `docs/adr/`.

ID	Titre	Statut	Résumé
ADR-1	Utilisation de Keycloak	Accepté	Centralisation de l'authentification et gestion des rôles (RBAC) via OAuth2/OIDC.
ADR-2	Utilisation de Kafka	Accepté	Communication asynchrone entre microservices via des topics dédiés.
ADR-3	Utilisation de PostgreSQL	Accepté	Base de données relationnelle par microservice, avec transactions ACID.

TABLE 2.1 – Tableau récapitulatif des ADR acceptées.

Note : Les ADR complètes sont disponibles dans le dossier `docs/adr/` sous forme de fichiers Markdown (format MADR).

Chapitre 3

Schémas de Base de Données

Diagramme de Classes

Ce diagramme montre les classes principales et leurs relations (héritage, associations). Il est utile pour comprendre la structure objet du système.

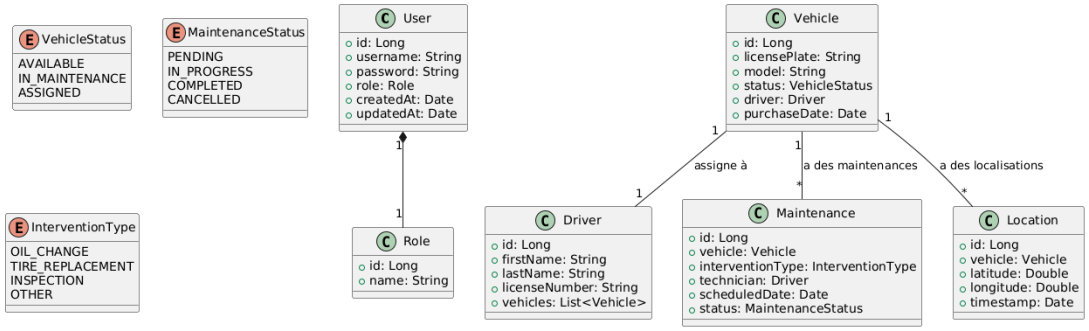


FIGURE 3.1 – Diagramme de Classe

Explications :

- **Classes principales :** User, Role, Vehicle, Driver, Maintenance, Location.
- **Relations :**
  - Un **User** a un **Role**.
  - Un **Vehicle** est assigné à un **Driver**.
  - Un **Vehicle** a plusieurs **Maintenance** et **Location**.
- **Énumérations :** VehicleStatus, MaintenanceStatus, InterventionType.

3.1 PostgreSQL

Les schémas de base de données sont conçus pour chaque microservice. Voici les tables principales :

3.1.1 Table `vehicles`

Colonne	Type	Description
id	SERIAL (PK)	Identifiant unique du véhicule.
license_plate	VARCHAR(20) (UNIQUE)	Plaque d'immatriculation.
model	VARCHAR(100)	Modèle du véhicule.
status	VARCHAR(20)	Statut (`available`, `in_maintenance`, `assigned`).
driver_id	INTEGER (FK)	ID du conducteur affecté (nullable).
purchase_date	DATE	Date d'achat du véhicule.
created_at	TIMESTAMP	Date de création (auto-généré).
updated_at	TIMESTAMP	Date de mise à jour (auto-généré).

TABLE 3.1 – Schéma de la table `vehicles`.

3.1.2 Exemple de Requête SQL

```sql
CREATE TABLE vehicles (
    id SERIAL PRIMARY KEY,
    license_plate VARCHAR(20) UNIQUE NOT NULL,
    model VARCHAR(100) NOT NULL,
    status VARCHAR(20) DEFAULT 'available',
    driver_id INTEGER REFERENCES drivers(id) ON DELETE SET NULL,
    purchase_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_vehicles_license_plate ON vehicles(license_plate);
CREATE INDEX idx_vehicles_driver ON vehicles(driver_id);
```

3.1.3 Migrations avec Flyway

Les migrations sont gérées avec Flyway. Exemple pour la table `vehicles` :

```sql
-- Fichier : V1__Create_vehicles_table.sql
CREATE TABLE vehicles (...);
```

Note : Les fichiers SQL complets sont disponibles dans `infrastructure/postgres/`.

Chapitre 4

Contrats d'API

4.1 REST (OpenAPI/Swagger)

Chaque microservice expose une API REST documentée avec OpenAPI 3.0. Voici un extrait pour le microservice `vehicle-service` :

4.1.1 Endpoints Principaux

Endpoint	Méthode	Rôle	Description
`/vehicles`	GET	admin, manager	Liste tous les véhicules.
`/vehicles`	POST	admin	Crée un nouveau véhicule.
`/vehicles/{id}`	GET	admin, manager, technicien	Récupère un véhicule par ID.
`/vehicles/{id}`	PUT	admin	Met à jour un véhicule.
`/vehicles/{id}/assign`	POST	admin, manager	Affecte un véhicule à un conducteur.

Table 4.1 – Endpoints de l'API `vehicle-service`.

4.1.2 Exemple de Contrat OpenAPI

```
openapi: 3.0.0
info:
  title: Vehicle Service API
  version: 1.0.0
paths:
  /vehicles:
    get:
      summary: Liste tous les v hicules
      security:
        - bearerAuth: [admin, manager]
      responses:
        '200':
          description: Liste des v hicules
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Vehicle'
```

4.1.3 GraphQL

Les schémas GraphQL sont définis pour chaque microservice :

Microservice	Fichier GraphQL	Types Principaux
vehicle-service	`vehicle-service.graphql`	`Vehicle`, `VehicleStatus`
conductor-service	`conductor-service.graphql`	`Driver`, `DriverRole`
maintenance-service	`maintenance-service.graphql`	`Maintenance`, `InterventionType`
localization-service	`localization-service.graphql`	`Location`

TABLE 4.2 – Schémas GraphQL par microservice.

4.1.4 Exemple : Schéma `vehicle-service`

```
type Vehicle {
  id: ID!
  licensePlate: String!
  model: String!
  status: VehicleStatus!
}

enum VehicleStatus {
  AVAILABLE
  IN_MAINTENANCE
  ASSIGNED
}

type Query {
  vehicles: [Vehicle!]!
}
```

Note : Les fichiers OpenAPI et GraphQL complets sont disponibles dans `contracts/`.

Chapitre 5

Configuration Keycloak

5.1 Étapes de Configuration

5.1.1 Création du Compte Admin

Un utilisateur administrateur a été créé dans le **realm 'master'** :

- **Identifiants** :
  - **Username** : `kenza_znt`
  - **Mot de passe** : `admin@`

5.1.2 Création du Realm `GestionFlotteM1`

Un realm dédié au projet a été créé pour isoler la configuration :

- **Nom du realm** : `GestionFlotteM1`
- **Objectif** : Centraliser la gestion des utilisateurs, rôles et clients OAuth2.

5.1.3 Définition des Rôles (RBAC)

Les rôles suivants ont été créés dans le realm `GestionFlotteM1` :

Rôle	Description
admin	Accès complet à toutes les fonctionnalités et données.
manager	Gestion des véhicules et des conducteurs (sans accès admin).
technicien	Accès limité aux fonctionnalités de maintenance.
utilisateur	Lecture seule (ex : consultation des véhicules disponibles).

Table 5.1 – Rôles RBAC configurés dans le realm `GestionFlotteM1`.

créer les Clients OAuth2 :

Client	Type	Flow	Description
conductor-service	Microservice Backend	Client Credentials	Gestion des conducteurs.
event-service	Microservice Backend	Client Credentials	Agrégation d'événements.
location-service	Microservice Backend	Client Credentials	Localisation GPS en temps réel.
maintenance-service	Microservice Backend	Client Credentials	Gestion des maintenances.
vehicle-service	Microservice Backend	Client Credentials	Gestion des véhicules.
frontend	Application Web	Standard Flow + Direct Grant	Interface utilisateur (React/-Vue).

Table 5.2 – Clients OAuth2 configurés dans Keycloak.

5.1.4 Création d'un Utilisateur dans `GestionFlotteM1`

Un utilisateur administrateur a été créé pour les tests :

- **Identifiants** :
  - **Username** : `admin`

  - **Mot de passe** : `admin123`
- **Rôles attribués** : `admin`

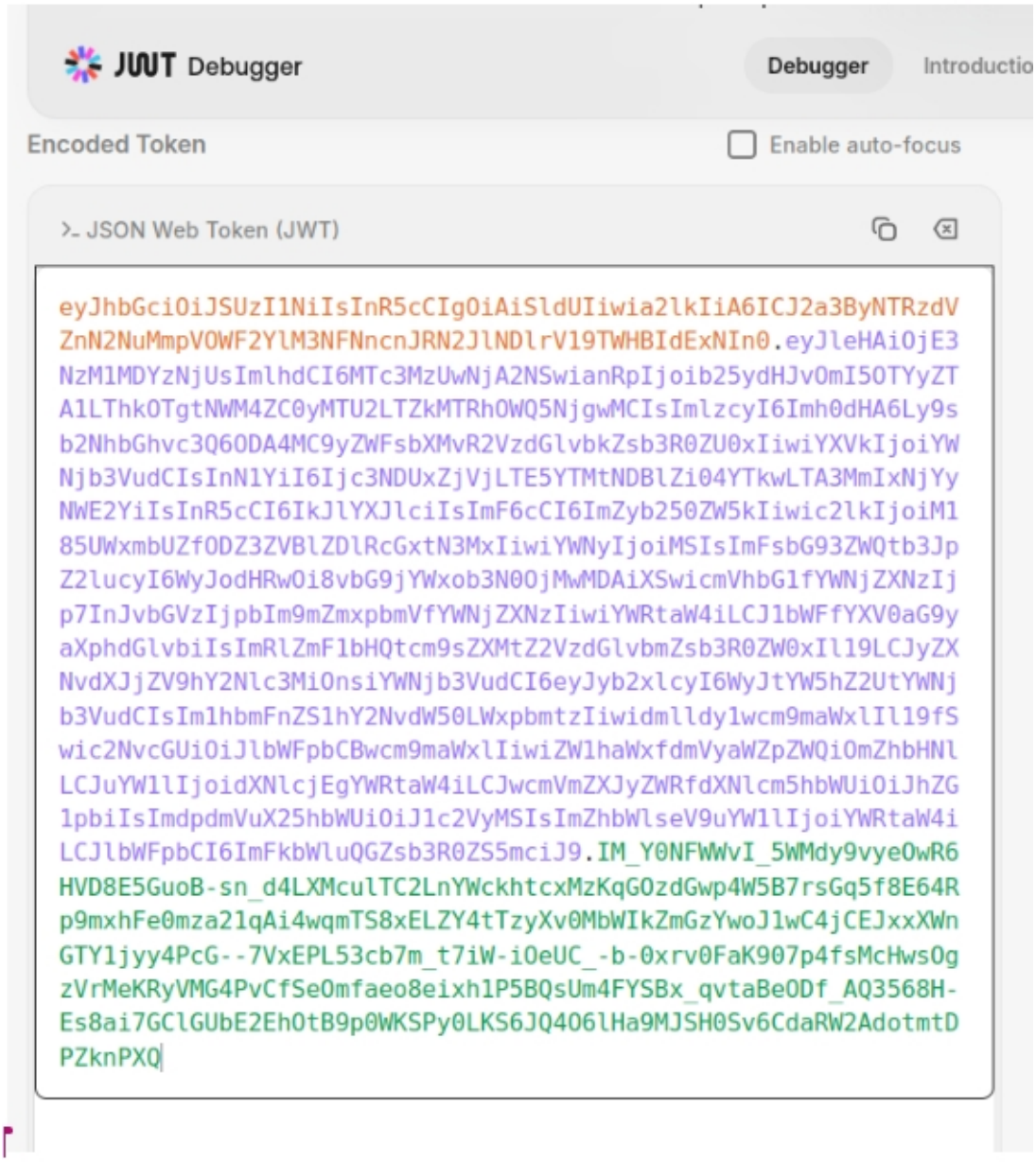
5.2 Test d'Authentification et Validation RBAC

5.2.1 Requête d'Obtention du Token (cURL)

```
curl -X POST "http://localhost:8080/realms/GestionFlotteM1/protocol/
   ↪ openid-connect/token" \
     -H "Content-Type: application/x-www-form-urlencoded" \
     -d "grant_type=password" \
     -d "client_id=frontend" \
     -d "client_secret=bQXsfUsQDbTSnFB7F5xMwveHzztO3Du1" \
     -d "username=admin" \
     -d "password=admin123"
```

5.2.2 Résultat et Décodage du Token

Le serveur Keycloak retourne un objet contenant l'access token qui sera utilisé par les microservices pour autoriser les requêtes.



Et quand on a décoder avec jwt.io on a eu ce résultat ce qui prouve que notre configuration keyclock fonctionne bien.

JSON Claims Breakdown


```
{
  "exp": 1773506365,
  "iat": 1773506065,
  "jti": "onrtro:b9962e05-8d98-5c8d-2156-6d14a9d96800",
  "iss": "http://localhost:8080/realms/GestionFlotteM1",
  "aud": "account",
  "sub": "77451f5c-19a3-40ef-8a90-072b16625a6b",
  "typ": "Bearer",
  "azp": "frontend",
  "sid": "3_9QlfmF_86wePed9Qplm7s1",
  "acr": "1",
  "allowed-origins": [
    "http://localhost:3000"
  ],
  "realm_access": {
    "roles": [
      "offline_access",
      "admin",
      "uma_authorization",
      "default-roles-gestionflottem1"
    ]
  },
  "resource_access": {
    "account": {
      "roles": [
        "manage-account",
        "manage-account-links",
        "view-profile"
      ]
    }
  },
  "scope": "email profile",
  "email_verified": false,
  "name": "user1 admin",
  "preferred_username": "admin",
  "given_name": "user1",
  "family_name": "admin",
  "email": "admin@flotte.fr"
}
```

Chapitre 6

Topics Kafka

6.1 Liste des Topics

Topic	Description	Producteur	Consommateur
vehicle-events	Événements du cycle de vie des véhicules.	vehicle-service	events-service
driver-events	Événements liés aux conducteurs.	conductor-service	events-service
maintenance-alerts	Alertes et événements de maintenance.	maintenance-service	events-service, vehicle-service
location-updates	Mises à jour GPS en temps réel.	localization-service	events-service, vehicle-service
system-notifications	Notifications système pour le frontend.	events-service	frontend (via WebSocket)

TABLE 6.1 – Topics Kafka configurés.

6.2 Exemple de Format de Message (JSON)

Topic `maintenance-alerts`

```
{
  "event_type": "maintenance_scheduled",
  "maintenance_id": 1,
  "vehicle_id": 1,
  "intervention_type": "oil_change",
  "technician_id": 1,
  "scheduled_date": "2026-03-15T09:00:00Z",
  "status": "pending",
  "timestamp": "2026-03-14T12:00:00Z"
}
```

Topic `vehicle-events`

```
{
  "event_type": "vehicle_created",
  "vehicle_id": 1,
  "license_plate": "AB123CD",
  "model": "Peugeot 3008",
  "status": "available",
  "driver_id": null,
  "timestamp": "2026-03-14T12:00:00Z"
}
```

6.2.1 Script pour Créer les Topics

Le script pour créer les topics est disponible dans :

`infrastructure/kafka/create_topics.sh`

Chapitre 7

Conclusion

7.1 Bilan de la Semaine 1

Les objectifs de la semaine 1 ont été atteints :

- **Architecture** : Diagrammes C4 et ADR documentés.
- **Base de données** : Schémas PostgreSQL et migrations Flyway.
- **Contrats d'API** : OpenAPI/Swagger et GraphQL définis.
- **Keycloak** : Realm configuré avec clients, rôles et utilisateurs.
- **Kafka** : Topics et formats de messages spécifiés.

Chapitre 8

Annexes

8.1 Livrables

- **Diagrammes C4** : Fichiers `.png` dans `docs/diagrams/`.
- **ADR** : Fichiers Markdown dans `docs/adr/`.
- **Schémas PostgreSQL** : Fichiers `.sql` dans `infrastructure/postgres/`.
- **Contrats OpenAPI** : Fichiers `.yml` dans `contracts/rest/`.
- **Schémas GraphQL** : Fichiers `.graphql` dans `contracts/graphql/`.
- **Configuration Keycloak** : Fichier `realm-export.json` dans `infrastructure/keycloak/`.
- **Topics Kafka** : Documentation dans `docs/kafka/`.